Accelerating Decompose-and-Then-Integrate Video Temporal Grounding

HSCC · NCHC Open Hackathon 2026 · Final Presentation

OpenACC

More Science, Less Programming

HSCC



Yu-Chee Tseng
NYCU



Cheng-Kuan Lin
NYCU



Pin-Hsuan Chou
NYCU



Yu-Ting Tseng
NYCU



Ching-Yuan Su
NYCU



Xin-Min Zhuang
NYCU



Xin-Zhe Xie
NYCU



Team Mentor: Claire Chang
NVIDIA

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

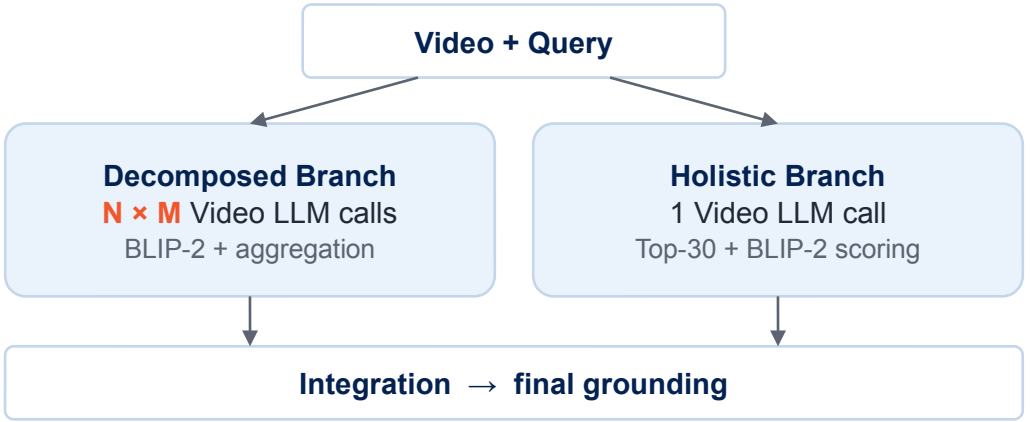
What We Accelerate

A training-free layer on a frozen Video LLM: more accurate, but many more model calls

Video Temporal Grounding

Video + “A person picks up a cup, then drinks” → which seconds?

Algorithmic motif: thousands of independent 7B Video LLM calls (30-beam search) + BLIP-2 scoring



Two independent branches, joined only at the end

Clip-Query Reasoning Grid

	Clip 1	Clip 2	Clip 3	Clip 4
Sub-q 1	call	call	call	call
Sub-q 2	call	call	call	call
Sub-q 3	call	call	call	call

≈17

Video LLM calls per query (QVHighlights avg.), instead of 1

Every cell is independent: a natural fit for parallel GPU work

25,307

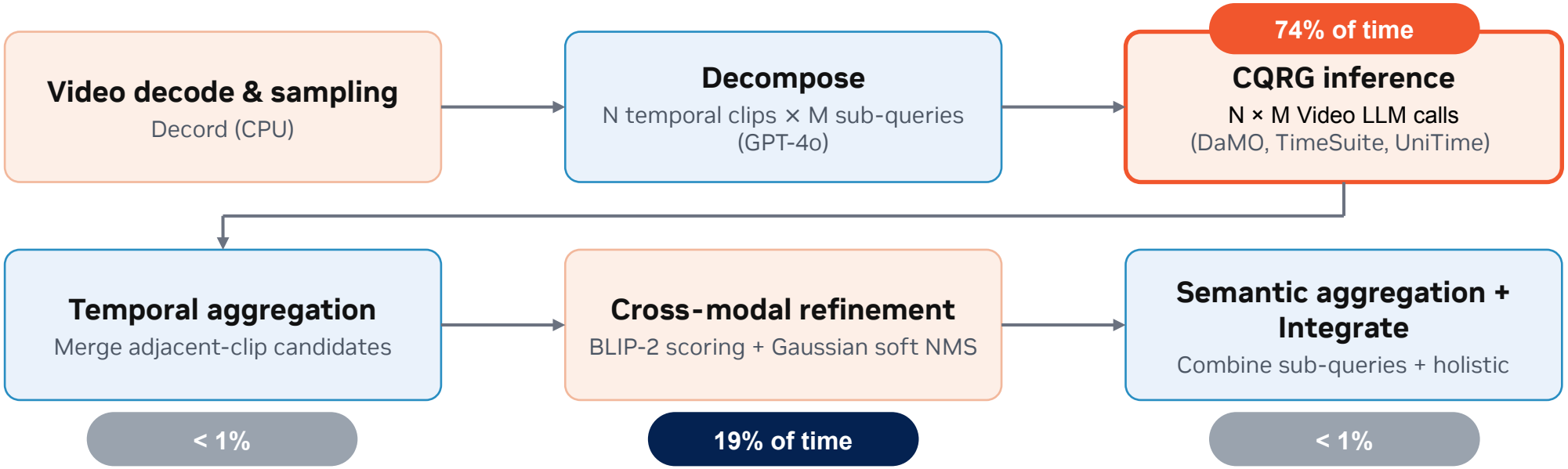
clip × subquery calls, full test set

43 h

per evaluation on one H200

Our Pipeline and Where the Time Goes

Time shares from the full test set: model calls dominate, and the GPU mostly waits



66–81%

of the time the GPU is idle (one process)

6.3 M

kernels in 10 videos, 80% under 10 µs

6%

holistic branch: one call per query

Acceleration targets Lightweight steps

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Evolution and Strategy

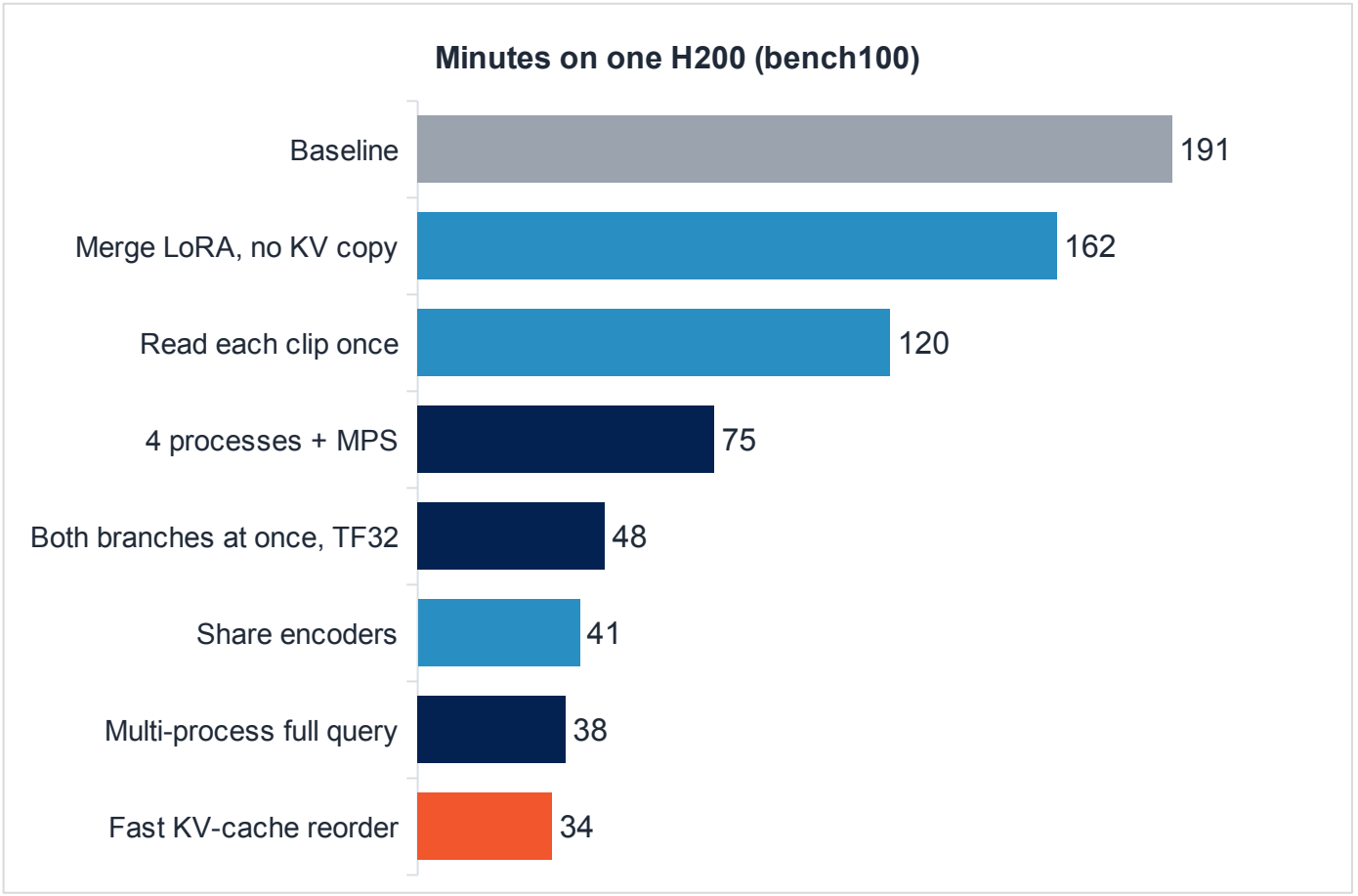
Goal: a full evaluation fast enough to rerun every day, without losing accuracy

Approach		What we found and did
Profile with Nsight	Done	GPU idle 66–81%: the real problem was waiting
Batch the N × M calls	Tried	0.53× with padding; equal-length batches +1.6%
Multi-stream	Changed	4 processes per GPU with MPS instead
GPU video decoding	Changed	Read each clip once: 75% of reads were repeats
Multi-GPU	Done	20 independent single-GPU jobs: 28 min
FP8 / TensorRT	Skipped	Would change model outputs
CUDA Graphs	Next	Many tiny kernels and per-step syncs remain

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

What We Did: Three Principles

bench100 on one H200: 191 → 34 min, accuracy checked after every step



Remove repeated work

re-reads, LoRA, KV copies

Fill the GPU

processes, MPS, parallel branches

Fix slow kernels

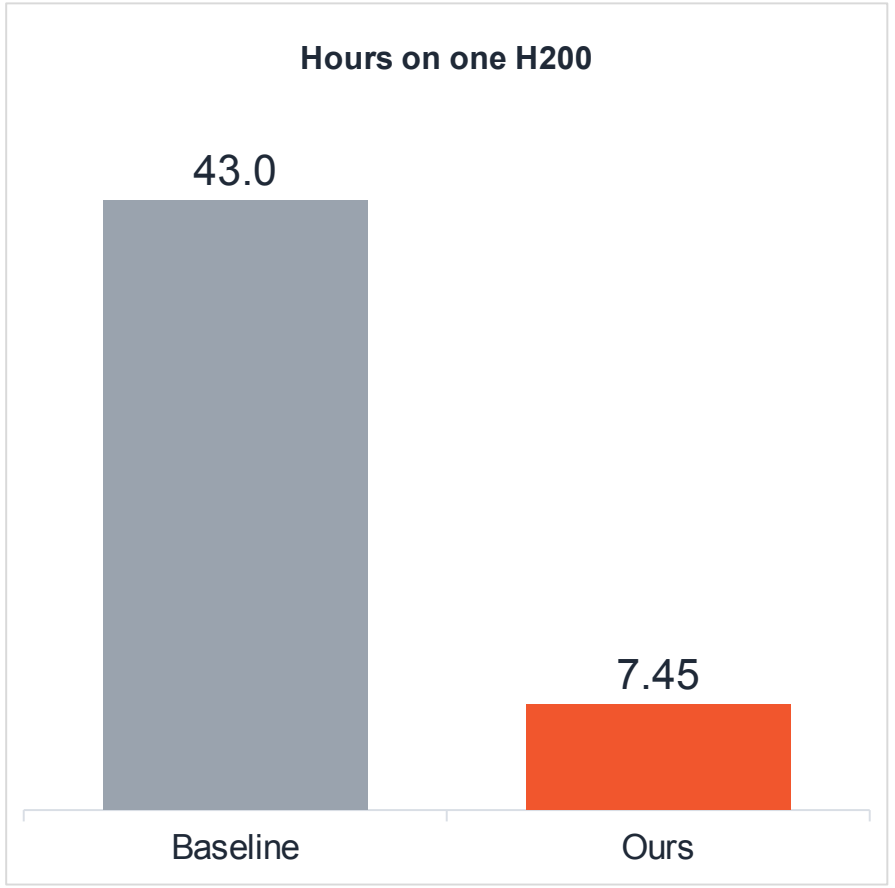
found with ncu

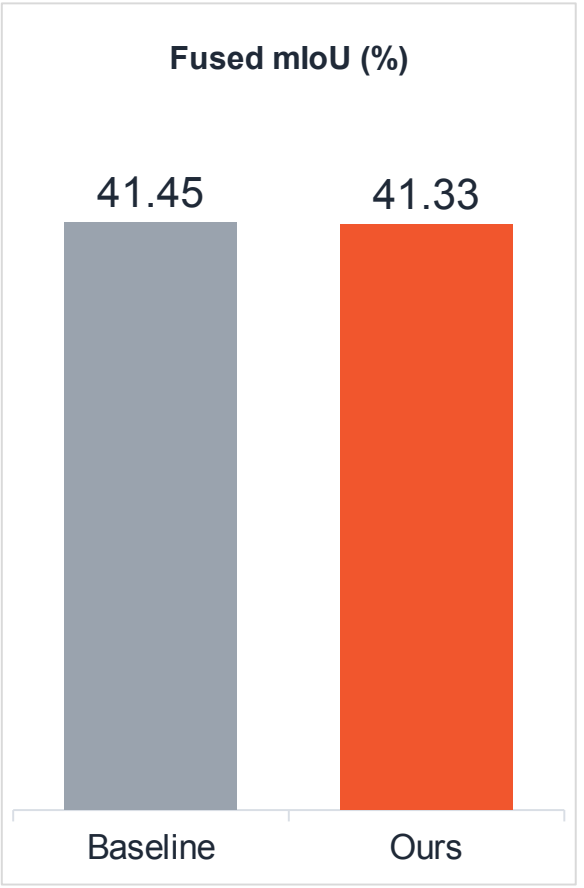
5.6× on bench100

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Results: 5.8× on One H200

Full test set (1,529 videos), same GPU, accuracy within 0.4%





Hardware vs software, on bench100

RTX 4090 ran only bench100

1.12×

Faster GPU alone: RTX 4090 → H200 (baseline code)

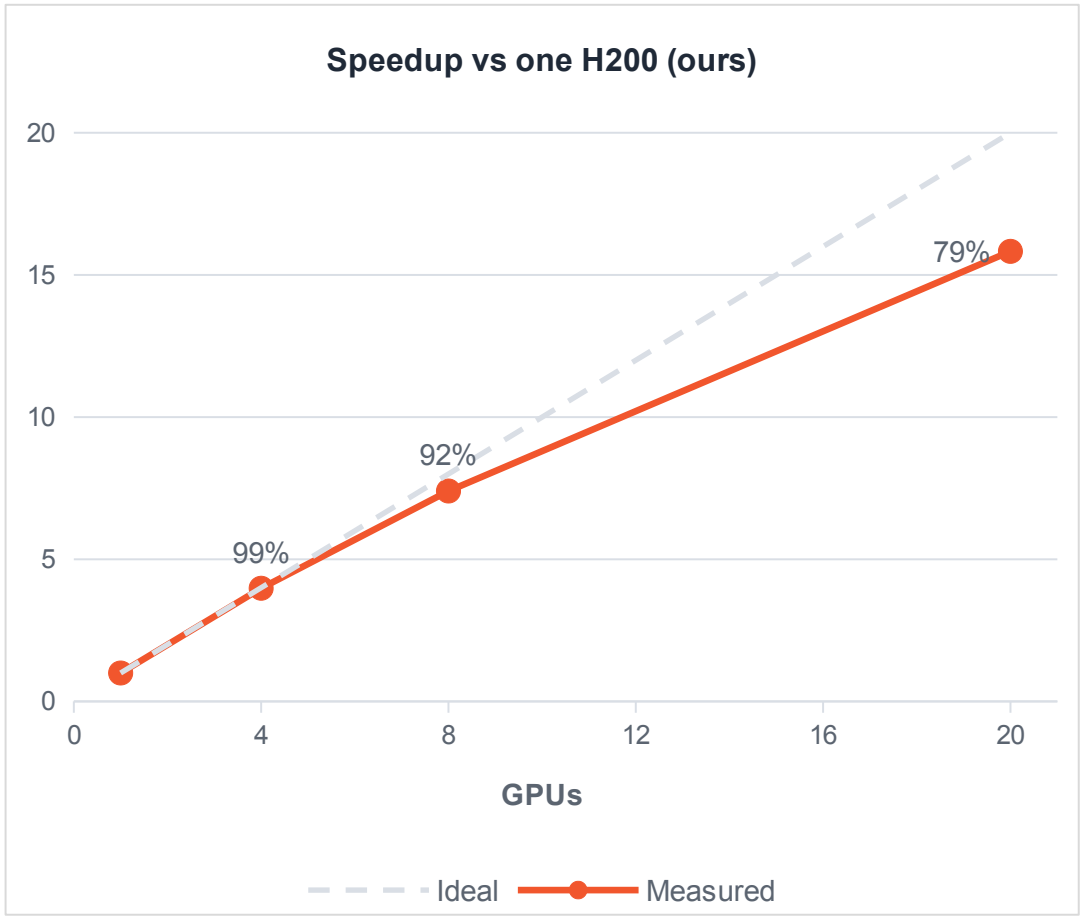
5.6×

Our software on the same H200

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Strong Scaling: 28 Minutes on 20 H200s

Same workload split into N independent single-GPU jobs, then merged



1,529 videos

20 jobs × 1 GPU, spread over several nodes

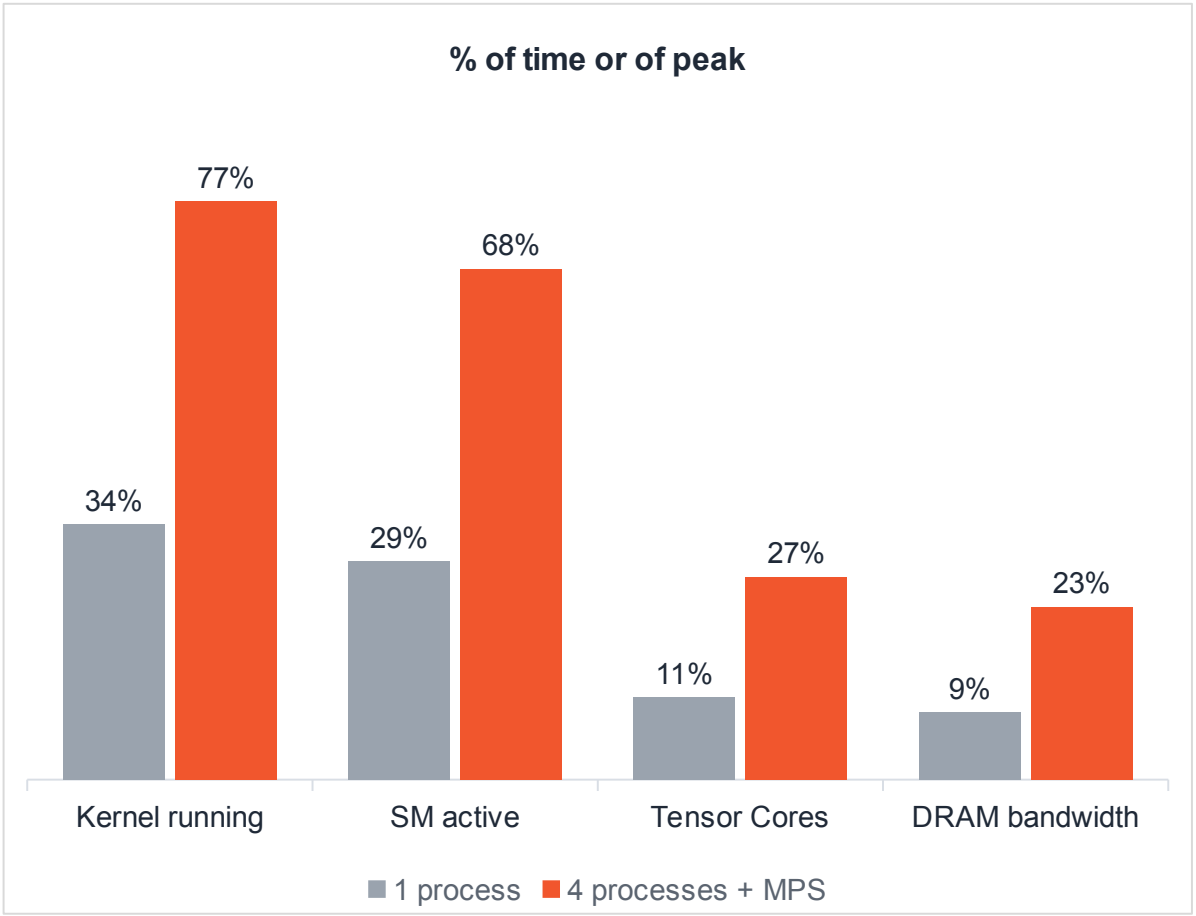
Merge and re-score: same result as one run

91×	vs the baseline on one GPU (43 h → 28 min)
4.75×	vs the baseline on the same 20 GPUs
231 s	start-up per job: why efficiency drops to 79%

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Final Profile: Busy, But Not at Peak

Sub-query grounding on one H200 (nsys GPU metrics, bench10)



Slow KV-cache reorder

4.3× faster per step; +9% end to end, same outputs

Prefill runs 30×

≈20% of generate time is repeated prefill: next step

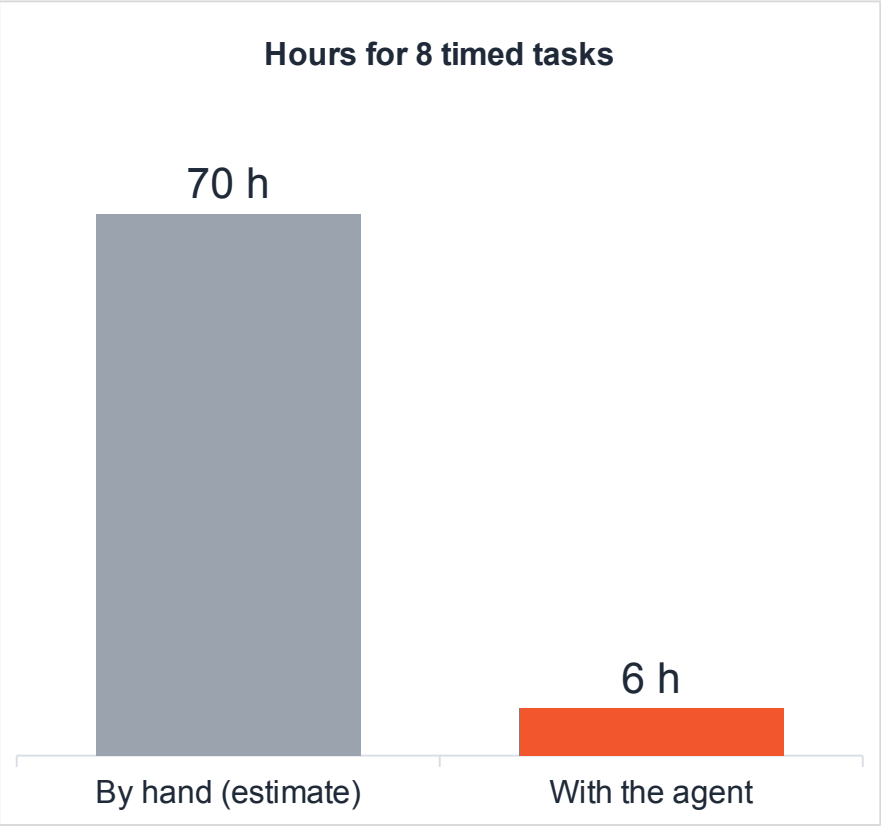
GEMMs

+0.6% from cuBLASLt: cuBLAS already picks the same kernels

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

AI Agent: Workflow Speedup

Claude Code builds tools and tests; people send GPU jobs and review



≈11× (10–13×). Hand-time estimates by the agent, reviewed by us; no control group.

218 Slurm jobs, 194 GPU-hours (team)

53 commits: 36 tools, 16 job scripts

14 CPU mock test suites before any GPU run

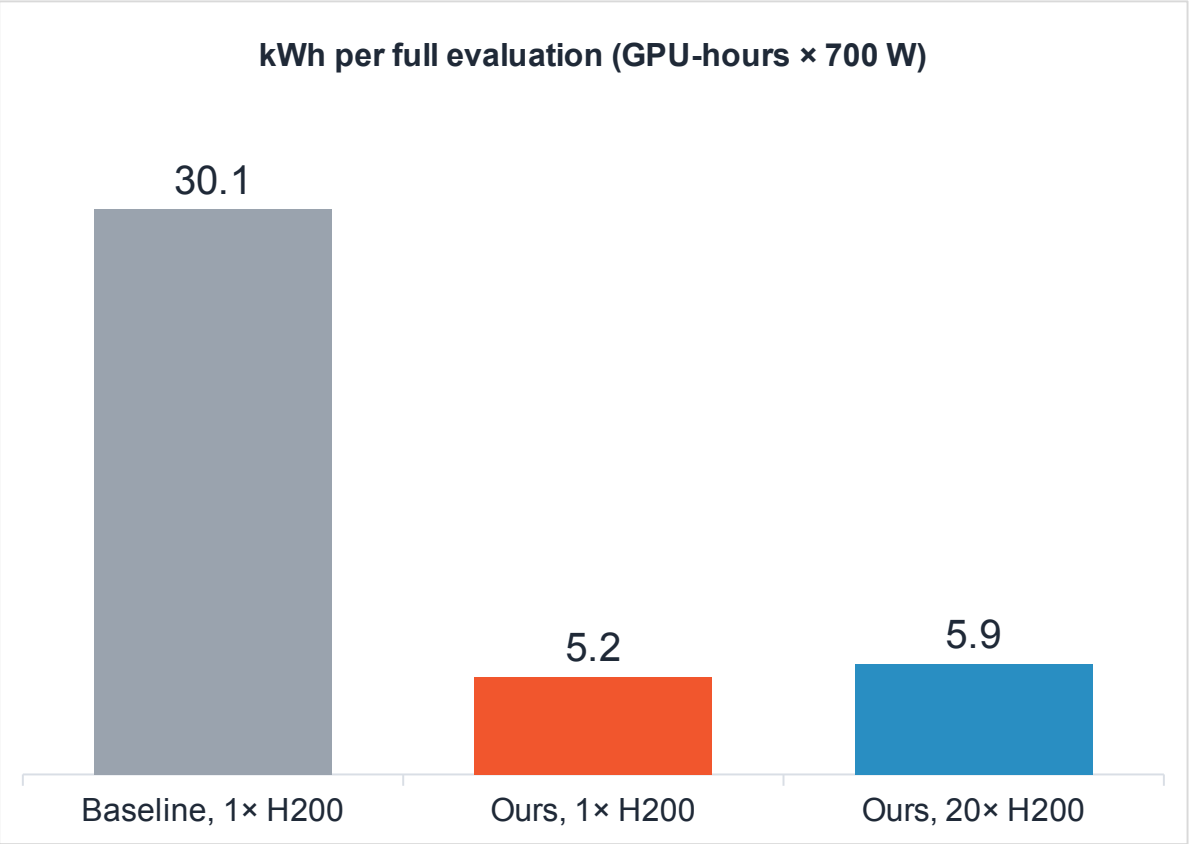
Lesson

Agents write tools fast, but make mistakes too: our checks caught an import bug and pairs silently dropped on out-of-memory.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Energy Efficiency

Same work in one-sixth of the GPU-hours



−83%

GPU energy per evaluation (upper-bound estimate)

+0.7 kWh

to use 20 GPUs: shorter wait, almost the same energy

Baseline GPUs often sit idle and draw less, so the real saving is smaller. The official calculator compares CPU-only nodes; our baseline already runs on a GPU.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

What Problems Have We Encountered?

Most of the work was finding what the original code hid

1	**Legacy app**	Each clip re-read for every sub-query	→ **Read once, reuse**
2	**Algorithms**	Padded batches: only 0.53× the speed	→ **Equal lengths: only +1.6%, not adopted**
3	**App bug**	Dropout left on at inference: no two runs agree	→ **Found by stage-by-stage replay**
4	**Tool limits**	Deterministic mode missed it; profiles mix MPS processes	→ **Built our own replay and compare tools**
5	**System setup**	4 h / 20-job limit; a node with a stray GPU process	→ **Split into segments; exclude the node**

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Wishlist

What would have saved us the most time

1 Tools

Profiler views per MPS process; a check for random ops in eval mode

2 Libraries

Beam search that prefills once and reorders the KV cache in place

3 Systems

Job arrays and node health checks on the dev partition

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Final Thoughts

Worth it: one full evaluation went from two days to under 30 minutes

Workload speedup (GPU)

5.8× on one H200; **91×** with 20 H200s

Workflow speedup (AI agent)

~9 working days of work in **~6 hours**

1 What we learned

Profile first: the GPU was waiting, not slow; a busy GPU is not a full one

2 What is new

A faster beam-search KV reorder; found the dropout bug that makes runs differ

3 Will we continue?

Yes: prefill once for 30 beams, CUDA Graphs, faster start-up per job

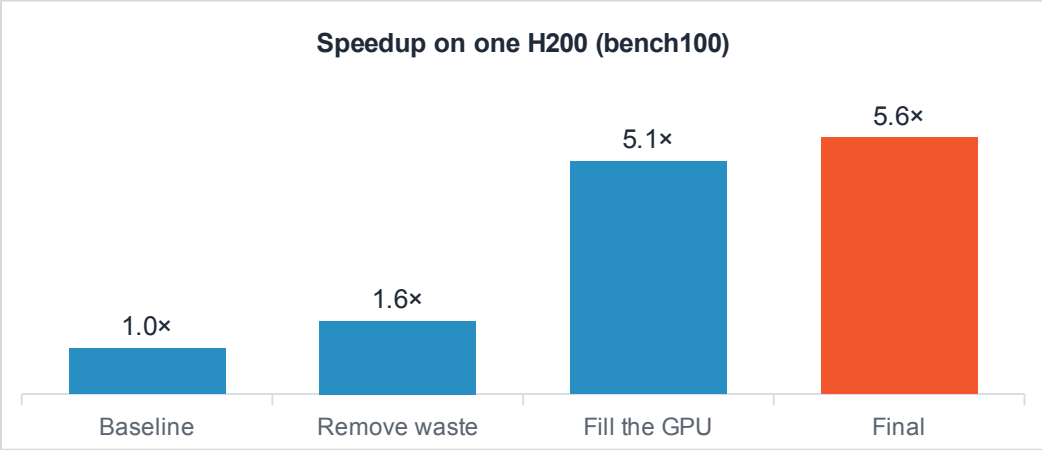
Thank you: **NCHC for the H200 GPU hours, and our mentor Claire Chang (NVIDIA) for her guidance**

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA


Thank You
OpenACC
More Science, Less Programming

Application Background

DaMO is a video–audio LLM that finds when a sentence happens in a video (video temporal grounding).
Decompose-and-Then-Integrate splits each query into sub-queries and 30 s clips, grounds each pair, and fuses the results.
Motifs: LLM inference with 30-beam search; video/audio encoders; BLIP-2 frame scoring.
Baseline: 43 GPU-hours for one evaluation of the QVHighlights test set on an H200.



Each step keeps accuracy within the baseline range; full test set: 5.8×.

Hackathon Objectives and Approach

Goal: full-test-set evaluation fast enough to iterate several times a day, without losing accuracy.
Profiling: nsys / ncu showed the GPU idle 66–81% (CPU decode, kernel launches).
Refactoring: read each clip once, share encoders, merge LoRA weights.
Programming model: multi-process workers with CUDA MPS; both branches run at once.
Tuning: TF32 for BLIP-2; fast KV-cache reorder for beam search.
Scaling: test set split into independent single-GPU jobs.

Technical Accomplishments and Impact

Speedup: 5.8× on one H200 (43.0 h → 7.45 h); 91× with 20 H200s (28 min); 6.3× vs the original RTX 4090 run.
Accuracy: fused mIoU within 0.4% of baseline.
Also: found an inference-dropout bug that made runs non-reproducible.
Why it matters: researchers can test an idea several times a day instead of once every two days, with ~83% less GPU energy per run.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

讓 AI 在影片裡找片段，快 5.8 倍



HSCC 團隊來自曾煜棋老師帶領的 HSCC 實驗室。原本估計需約 9 個工作天的 profiling 工具開發與驗證工作，因為有 AI Agent，約 6 小時完成（約 11 倍）；利用 GPU 將影片時序定位的評估加速了 5.8 倍，20 張 H200 平行時達 91 倍！！

影片時序定位是讓 AI 根據一句話，找出影片中對應片段的開始與結束時間，可用於影片搜尋、精華剪輯與監視影像檢索。我們加速的 DaMO 方法把查詢拆成子查詢、把影片切成 30 秒片段，再用影音大型語言模型逐一判斷，準確度較高，但完整測試集光是片段 × 子查詢就要呼叫模型兩萬五千多次，單張 H200 需要 43 小時。我們用 Nsight 分析，發現 GPU 有六到八成時間在等 CPU 解碼影片或啟動小 kernel，於是讓同一片段只讀一次、合併模型權重、在一張 GPU 上同時跑多個程序，並修正 beam search 搬移 KV cache 太慢的問題。最後單張 GPU 只要 7.45 小時，20 張 GPU 只要 28 分鐘，準確度差距在 0.4% 以內，每次評估的 GPU 用電也少了約八成。

Table 1: Full test set (1,529 videos), wall-clock time on H200

H200 GPUs	*Time*	*vs 1 GPU*	*Efficiency*	*vs Baseline*
Baseline, 1	43.0 h	—	—	**1×**
Ours, 1	7.45 h	1.00×	100%	**5.8×**
Ours, 4	1 h 52 m	3.98×	99%	**23×**
Ours, 8	60 m	7.40×	92%	**43×**
Ours, 20	28 m	15.8×	79%	**91×**

Backup: Every Change, Measured

bench100 on one H200; every step passed the accuracy gate (fused mIoU ≥ 0.4755)

#	Change	Flag	Time	Speedup	Output check
0	Baseline (mean of 5 runs)	—	11,471 s	1.00×	fused mIoU 0.4805–0.4927
1	Merge LoRA into weights; no KV head copy	--merge_tg_lora --no_repeat_kv	9,719 s	1.18×	gate PASS
2	Read and encode each clip once	--clip_cache	7,186 s	1.60×	same function, reused input
3	4 sub-query processes per GPU with MPS	--num_workers 4 --mps	4,510 s	2.54×	gate PASS
4	Both branches at once; BLIP-2 in TF32	PARALLEL_BRANCHES=1, --blip_precision tf32	2,849 s	4.03×	gate PASS
5	Share encoder outputs across sub-queries	--share_encoder	2,466 s	4.65×	gate PASS
6	Multi-process full-query steps (v1)	--num_workers (A1, A2)	2,250 s	5.10×	3 runs, gate PASS
7	Fast KV-cache reorder (v2)	--fast_reorder	2,046 s	**5.61×**	same outputs (replay)

Full test set (1,529 videos): 43.0 h → 7.45 h on one H200 (5.8×); merged 4/8/20-segment runs give the same results as one run.

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Backup: Tried but Not Adopted

We kept only changes that keep the model as is and pass the accuracy gate

Idea	Result	Why
Batch generate calls (with padding)	0.53× (slower)	FlashAttention 2 re-gathers the KV cache every step when inputs are padded
Batch only equal-length prompts	+16% alone, +1.6% in the final setup	GPU already busy most of the time; the limit is small kernels and per-step CPU syncs, not batch size
Batches of 4+ prompts	Some outputs differ; out of memory	30 beams × 4 = 120 sequences per call
6 or 8 processes per GPU	Not run (memory estimate)	≈23.8 GB per process (peak on RTX 4090); 6 would need ≈143 GB of 141 GB
Turn off inference dropout (bug fix)	Two runs give exactly the same output	Changes model outputs vs the baseline; we will report it to the DaMO authors
Prefill once, share KV across 30 beams	Confirmed Nov 3: prefill M = 30 × prompt (≈15,000); ≈20% of generate time	Changes the numeric path; needs full revalidation (next step)
cuBLASLt algorithm search for GEMMs	+0.6% of GEMM time (best per shape)	On Hopper, cuBLAS already uses cuBLASLt kernels; decode is memory-bound, prefill at 65% of peak

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Backup: The Method

Decompose-and-Then-Integrate on a frozen Video LLM

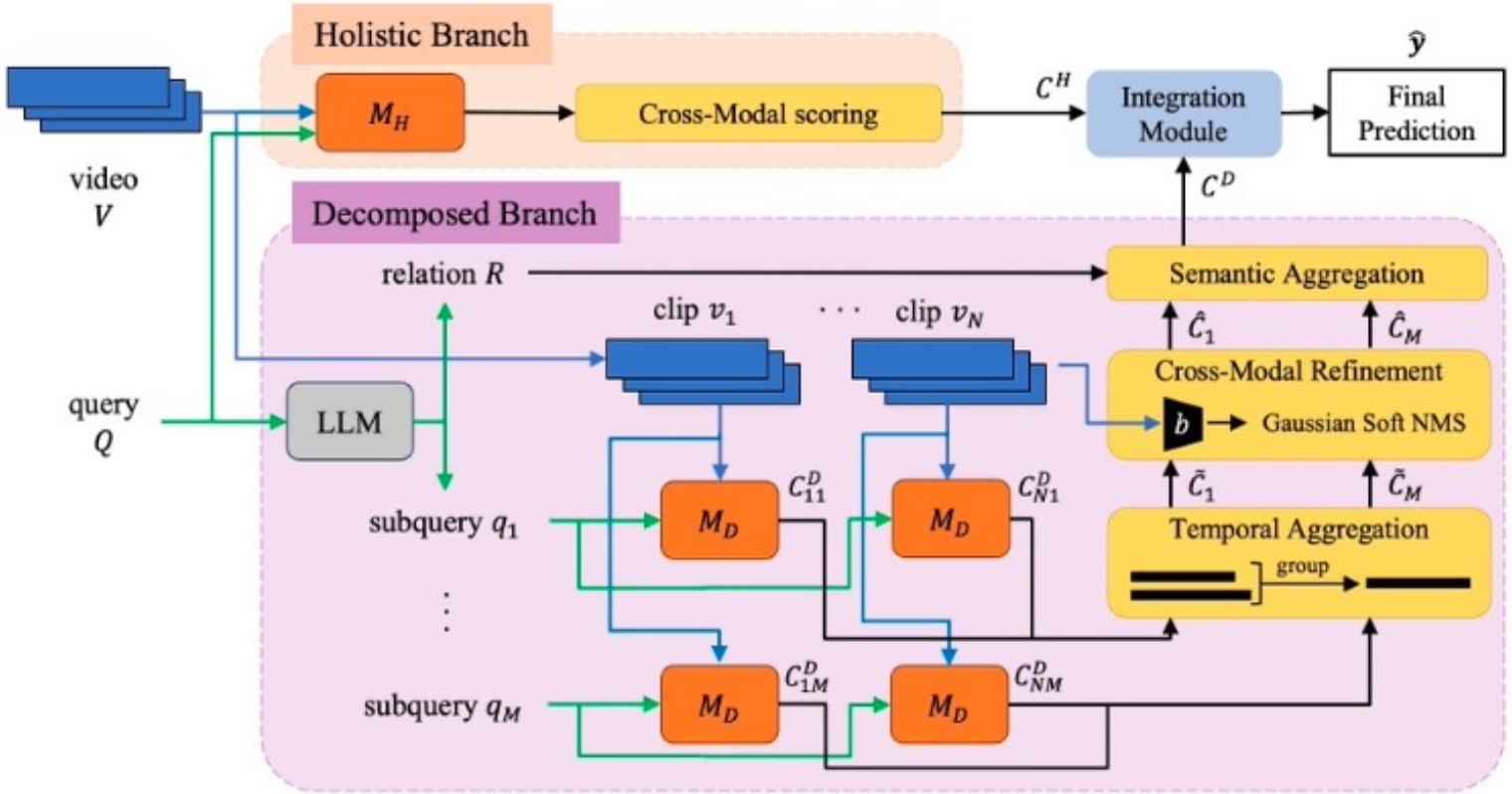


Figure 2. The decompose-and-then-integrate framework.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Backup: Accuracy in Detail

Full test set (1,529 videos, 1,542 queries); fused result after integration

Run	mIoU	R@0.3	R@0.5	R@0.7	1-GPU time
Baseline, run 1	41.47	57.85	40.73	24.06	43.3 h
Baseline, run 2	41.42	57.85	40.47	24.19	42.7 h
Ours, 4 segments	41.30	57.39	40.47	24.06	**7.45 h**
Ours, 8 segments	41.38	57.46	40.40	24.12	7.81 h
Ours, 20 segments	41.31	57.65	40.40	23.61	8.48 h

- Gap to the baseline: at most 0.17 mIoU points (0.4%); R@0.5 is the same.
- The original code is not reproducible run to run (inference dropout), so we compare ranges, not single runs.

48.1–49.3 baseline mIoU on bench100 over 5 runs

≥ 47.55 our accuracy gate on bench100 (lowest run − 0.5)

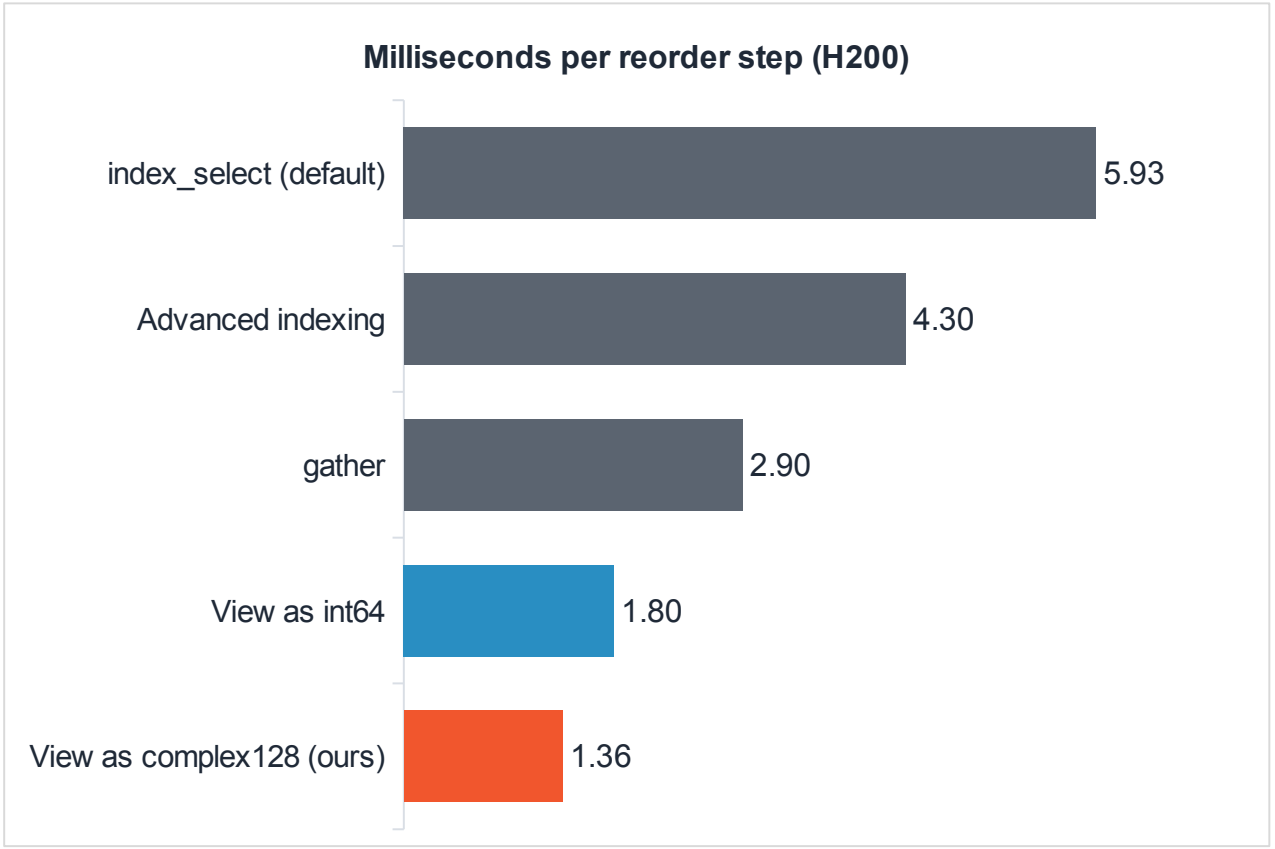
17 s per query on one H200 (was 100 s)

1.1 s per query with 20 H200s

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Backup: Fast KV-Cache Reorder

Beam search reorders the KV cache 64 times per step (32 layers × K and V)



13% of HBM bandwidth used by the default copy (≈620 GB/s)

4.3× faster per step: same bytes, 8× fewer elements

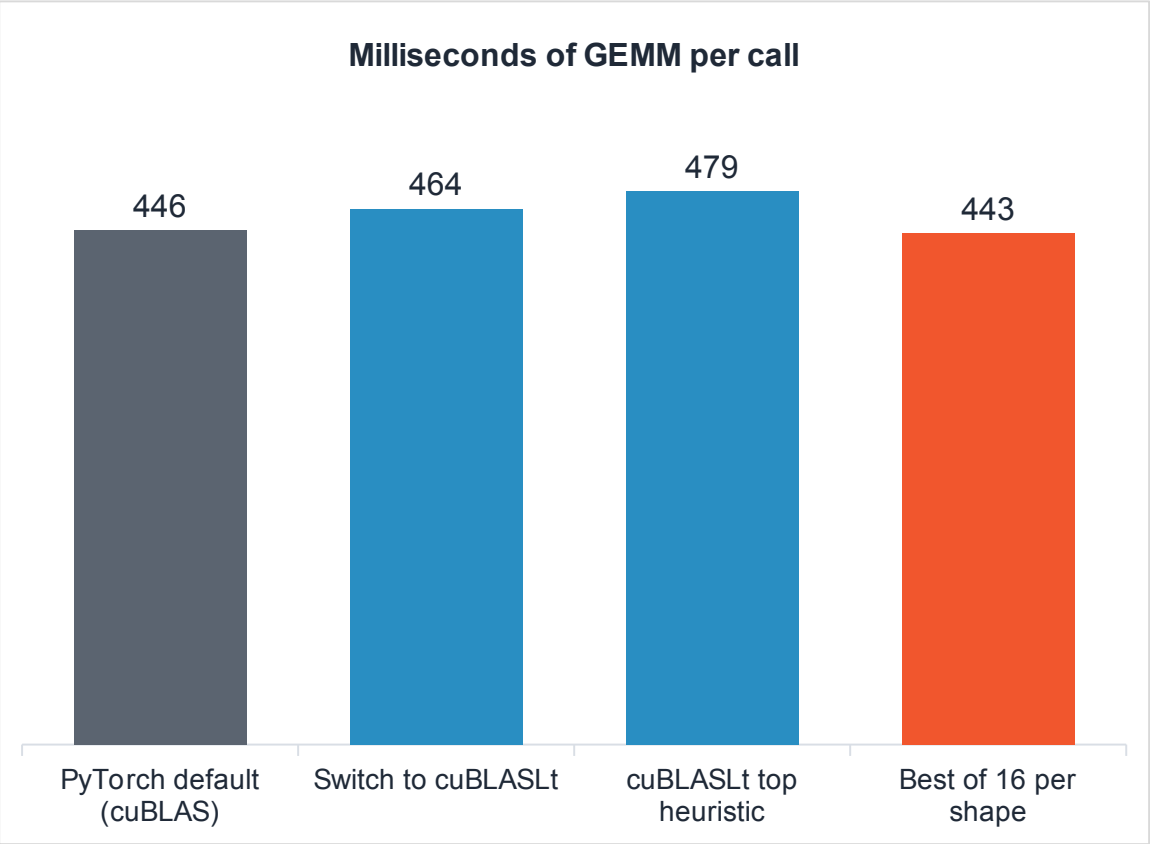
Same outputs on 125 replayed pairs, 3,306 beam steps

+9% end to end on bench100 (2,250 → 2,046 s)

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Backup: GEMMs and cuBLASLt

GEMM time per sub-query call, weighted by how often each shape runs



+0.6% — best case: cuBLAS already picks the same Hopper kernels

42–76% — of HBM bandwidth in decode (M = 30): memory-bound

65% — of peak FLOPS in prefill (M ≈ 15,000): compute-bound

30× — the same prefill runs once per beam: the real saving

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA

Backup: Why Not Multi-Stream, vLLM or FP8?

Each choice kept the model as is and the code change small

Multi-stream

Why MPS instead

- generate() depends on the previous step
- Python threads are limited by the GIL
- 4 processes + MPS: GPU busy 34% → 77%
- Cost: each process loads its own model (≈23.8 GB)

vLLM / TensorRT-LLM

Why not

- DaMO is a custom video–audio model with its own encoders and fusion
- Not supported by these engines
- A new engine changes the numerics and the outputs

FP8 / quantization / fewer beams

Why not

- All of them change the model's outputs
- Our rule: keep the model, only faster
- Only TF32 for BLIP-2, which passes the accuracy gate

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Backup: Strong Scaling Numbers

Each segment is one Slurm job on one H200; times are wall-clock seconds

GPUs	Sum of segments	Longest segment	Speedup	Efficiency	vs baseline
1	26,818 s	—	1.00×	100%	5.8×
4	26,818 s	6,741 s	3.98×	99%	23×
8	28,133 s	3,626 s	7.40×	92%	43×
20	30,523 s	1,694 s	15.8×	79%	**91×**
20 (baseline)	154,817 s	7,867–8,233 s	—	—	1×

- 1-GPU time = sum of the 4 segments (the run is too long for the 4 h job limit).
- Same 20 GPUs: baseline 2 h 14 m vs ours 28 min (4.75×).

231 s — fixed start-up per segment (fit: 231 + 25,895 ÷ N)

1 load — per GPU is the minimum: speed up loading, not remove it

1,526 / 1,528 s — mean segment time of the two submission batches

20 — jobs per user on the dev partition: our limit

OpenACC More Science, Less Programming · OPEN HACKATHONS · NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing · NVIDIA